

1.Loading the CSV

In our Jupyter Notebook, load the Telco CSV into a DataFrame.

```
[21]: import pandas as pd

df = pd.read_csv("telco_customer_churn.csv")
df
```

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService	OnlineSecurity	...	DeviceProtection
0	7590-VHVEG	Female	0	Yes	No	1	No	No phone service	DSL	No	...	No
1	5575-GNVDE	Male	0	No	No	34	Yes	No	DSL	Yes	...	Yes
2	3668-QPYBK	Male	0	No	No	2	Yes	No	DSL	Yes	...	No
3	7795-CFOCW	Male	0	No	No	45	No	No phone service	DSL	Yes	...	Yes
4	9237-HQITU	Female	0	No	No	2	Yes	No	Fiber optic	No	...	No
...	...	...	...	...	...	...	...	...	...	...	...	...
7038	6840-RESVB	Male	0	Yes	Yes	24	Yes	Yes	DSL	Yes	...	Yes
7039	2234-XADUH	Female	0	Yes	Yes	72	Yes	Yes	Fiber optic	No	...	Yes
7040	4801-JAZZL	Female	0	Yes	Yes	11	No	No phone service	DSL	Yes	...	No
7041	8361-LTMKD	Male	1	Yes	No	4	Yes	Yes	Fiber optic	No	...	No
7042	3186-AJIEK	Male	0	No	No	66	Yes	No	Fiber optic	Yes	...	Yes

TechSupport	StreamingTV	StreamingMovies	Contract	PaperlessBilling	PaymentMethod	MonthlyCharges	TotalCharges	Churn
No	No	No	Month-to-month	Yes	Electronic check	29.85	29.85	No
No	No	No	One year	No	Mailed check	56.95	1889.5	No
No	No	No	Month-to-month	Yes	Mailed check	53.85	108.15	Yes
Yes	No	No	One year	No	Bank transfer (automatic)	42.30	1840.75	No
No	No	No	Month-to-month	Yes	Electronic check	70.70	151.65	Yes
...	...	...	...	...	...	...	...	...
Yes	Yes	Yes	One year	Yes	Mailed check	84.80	1990.5	No
No	Yes	Yes	One year	Yes	Credit card (automatic)	103.20	7362.9	No
No	No	No	Month-to-month	Yes	Electronic check	29.60	346.45	No
No	No	No	Month-to-month	Yes	Mailed check	74.40	306.6	Yes
Yes	Yes	Yes	Two year	Yes	Bank transfer (automatic)	105.65	6844.5	No

Performing EDA

[22]: # First 5 rows

df.head()

[22]:	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService	OnlineSecurity	...	DeviceProtection	TechSupport
0	7590-VHVEG	Female	0	Yes	No	1	No	No phone service	DSL	No	...	No	No
1	5575-GNVDE	Male	0	No	No	34	Yes	No	DSL	Yes	...	Yes	No
2	3668-QPYBK	Male	0	No	No	2	Yes	No	DSL	Yes	...	No	No
3	7795-CFOCW	Male	0	No	No	45	No	No phone service	DSL	Yes	...	Yes	Yes
4	9237-HQITU	Female	0	No	No	2	Yes	No	Fiber optic	No	...	No	No
5 rows × 21 columns													

[23]: # Shape

df.shape

[23]: (7043, 21)

[24]: # Column names

df.columns

[24]: Index(['customerID', 'gender', 'SeniorCitizen', 'Partner', 'Dependents', 'tenure', 'PhoneService', 'MultipleLines', 'InternetService', 'OnlineSecurity', 'OnlineBackup', 'DeviceProtection', 'TechSupport', 'StreamingTV', 'StreamingMovies', 'Contract', 'PaperlessBilling', 'PaymentMethod', 'MonthlyCharges', 'TotalCharges', 'Churn'], dtype='str')

[25]:

```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 21 columns):
#   Column                Non-Null Count  Dtype
---  ---                ---
0   customerID            7043 non-null   str
1   gender                7043 non-null   str
2   SeniorCitizen         7043 non-null   int64
3   Partner               7043 non-null   str
4   Dependents            7043 non-null   str
5   tenure               7043 non-null   int64
6   PhoneService          7043 non-null   str
7   MultipleLines         7043 non-null   str
8   InternetService       7043 non-null   str
9   OnlineSecurity        7043 non-null   str
10  OnlineBackup          7043 non-null   str
11  DeviceProtection      7043 non-null   str
12  TechSupport           7043 non-null   str
13  StreamingTV           7043 non-null   str
14  StreamingMovies       7043 non-null   str
15  Contract              7043 non-null   str
16  PaperlessBilling      7043 non-null   str
17  PaymentMethod         7043 non-null   str
18  MonthlyCharges        7043 non-null   float64
19  TotalCharges          7043 non-null   str
20  Churn                 7043 non-null   str
dtypes: float64(1), int64(2), str(18)
memory usage: 1.1 MB
```

[26]:

```
# Data types
```

```
df.dtypes
```

[26]:

```
customerID      str
gender          str
SeniorCitizen   int64
Partner         str
Dependents      str
tenure          int64
PhoneService    str
MultipleLines   str
InternetService str
OnlineSecurity  str
OnlineBackup    str
DeviceProtection str
TechSupport     str
StreamingTV     str
StreamingMovies str
Contract        str
PaperlessBilling str
PaymentMethod   str
MonthlyCharges  float64
TotalCharges    str
Churn           str
dtype: object
```

```
[29]: # Data types
df.dtypes
```

```
[29]: customerID      str
gender            str
SeniorCitizen    int64
Partner          str
Dependents       str
tenure           int64
PhoneService     str
MultipleLines    str
InternetService  str
OnlineSecurity   str
OnlineBackup     str
DeviceProtection str
TechSupport      str
StreamingTV      str
StreamingMovies  str
Contract         str
PaperlessBilling str
PaymentMethod    str
MonthlyCharges   float64
TotalCharges     float64
Churn            str
dtype: object
```

```
[30]: df.isnull().sum()
```

```
[30]: customerID      0
gender           0
SeniorCitizen    0
Partner          0
Dependents       0
tenure           0
PhoneService     0
MultipleLines    0
InternetService  0
OnlineSecurity   0
OnlineBackup     0
DeviceProtection 0
TechSupport      0
StreamingTV      0
StreamingMovies  0
Contract         0
PaperlessBilling 0
PaymentMethod    0
MonthlyCharges   0
TotalCharges     11
Churn            0
dtype: int64
```

```
[32]: df[df["TotalCharges"].isna()][["tenure", "MonthlyCharges", "TotalCharges", "Churn"]]
```

```
[32]:
```

	tenure	MonthlyCharges	TotalCharges	Churn
488	0	52.55	NaN	No
753	0	20.25	NaN	No
936	0	80.85	NaN	No
1082	0	25.75	NaN	No
1340	0	56.05	NaN	No
3331	0	19.85	NaN	No
3826	0	25.35	NaN	No
4380	0	20.00	NaN	No
5218	0	19.70	NaN	No
6670	0	73.35	NaN	No
6754	0	61.90	NaN	No

```
[33]: df["TotalCharges"] = df["TotalCharges"].fillna(0)
```

```
[35]: df["TotalCharges"].isna().sum()
```

```
[35]: np.int64(0)
```

Train/Test Split

```
[36]: X = df.drop(columns=["customerID", "Churn"])
      y = df["Churn"]
```

```
[37]: # Quickly verify:
      print(X.shape)
      print(y.shape)
```

```
(7043, 19)
(7043,)
```

```
[38]: print(X.columns)
```

```
Index(['gender', 'SeniorCitizen', 'Partner', 'Dependents', 'tenure',
      'PhoneService', 'MultipleLines', 'InternetService', 'OnlineSecurity',
      'OnlineBackup', 'DeviceProtection', 'TechSupport', 'StreamingTV',
      'StreamingMovies', 'Contract', 'PaperlessBilling', 'PaymentMethod',
      'MonthlyCharges', 'TotalCharges'],
      dtype='str')
```

```
[39]: from sklearn.model_selection import train_test_split

      X_train, X_test, y_train, y_test = train_test_split(
          X,
          y,
          test_size=0.20,
          random_state=42,
          stratify=y
      )
```

```
[40]: print("X_train:", X_train.shape)
      print("X_test :", X_test.shape)
      print("y_train:", y_train.shape)
      print("y_test :", y_test.shape)
```

```
X_train: (5634, 19)
X_test  : (1409, 19)
y_train: (5634,)
y_test  : (1409,)
```

```
[15]: # And check the churn proportions:
```

```
[41]: print("Train:")
      print(y_train.value_counts(normalize=True))

      print("\nTest:")
      print(y_test.value_counts(normalize=True))
```

```
Train:
Churn
No      0.734647
Yes     0.265353
Name: proportion, dtype: float64
```

```
Test:
Churn
No      0.734564
Yes     0.265436
Name: proportion, dtype: float64
```

```
[42]: X_train.dtypes
```

```
[42]: gender                str
      SeniorCitizen        int64
      Partner              str
      Dependents           str
      tenure               int64
      PhoneService         str
      MultipleLines        str
      InternetService      str
      OnlineSecurity       str
      OnlineBackup         str
      DeviceProtection     str
      TechSupport          str
      StreamingTV          str
      StreamingMovies      str
      Contract             str
      PaperlessBilling     str
      PaymentMethod        str
      MonthlyCharges       float64
      TotalCharges         float64
      dtype: object
```

```
[43]: numeric_features = X_train.select_dtypes(
        include=["int64", "float64"]
    ).columns

categorical_features = X_train.select_dtypes(
        include=["object", "str"]
    ).columns

print("Numeric:", numeric_features.tolist())
print("Categorical:", categorical_features.tolist())

Numeric: ['SeniorCitizen', 'tenure', 'MonthlyCharges', 'TotalCharges']
Categorical: ['gender', 'Partner', 'Dependents', 'PhoneService', 'MultipleLines', 'InternetService', 'OnlineSecurity', 'OnlineBackup', 'DeviceProtection', 'TechSupport', 'StreamingTV', 'StreamingMovies', 'Contract', 'PaperlessBilling', 'PaymentMethod']
```

Create preprocessing pipeline

```
[45]: from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder

preprocessor = ColumnTransformer(
    transformers=[
        ("num", StandardScaler(), numeric_features),
        ("cat", OneHotEncoder(handle_unknown="ignore"), categorical_features)
    ]
)
```